Post-Training Weight Quantization in Small Language Models: Reasoning–Efficiency Trade-offs Across GGUF Precision Levels

Prakyath Chandran

MSc Applied AI and Data Science

code | LinkedIn

1. Abstract

Weight quantization can reduce the storage and computational requirements of language models, but it may also affect their reasoning capabilities. This study investigates whether quantization causes gradual or abrupt degradation in the reasoning performance of small language models. Three instruction-tuned models were evaluated: Qwen3-0.6B, Gemma 3 1B IT, Llama 3.2 1B Instruct. Each model was tested using a BF16 reference version and five GGUF quantization formats: Q8_0, Q6_K, Q5_K_M, Q4_K_M and Q3_K_M.

Evaluation was performed on the same fixed 50-question subset of the GSM8K mathematical reasoning dataset. Exact-answer accuracy, average latency, peak RAM usage, generation speed, and stored model size were recorded. The results show that quantization did not produce one universal degradation pattern. Accuracy changes varied across models and precision levels. Qwen Q4_K_M matched its BF16 accuracy, Gemma Q4_K_M slightly exceeded its BF16 result, and Llama Q6_K achieved the highest quantized accuracy for that model. However, Q3_K_M produced larger accuracy reductions for Qwen and Llama.

Quantization consistently improved storage and inference efficiency. These findings suggest that quantization provides practical efficiency benefits, but the effect on reasoning performance is model-dependent.

Keywords: weight quantization; small language models; GGUF; reasoning; llama.cpp; inference efficiency, GSM8K

2. Introduction

Small language models can provide useful text-generation and reasoning capabilities while requiring fewer computational resources than larger models. This makes them relevant for local and edge-device deployment, where memory capacity, storage, and inference speed are limited.

Weight quantization is a common method for reducing model storage and inference cost by representing model weights with lower numerical precision. Although quantization can improve efficiency, reducing precision may also affect model capabilities. Previous quantization research has demonstrated substantial compression and efficiency benefits, but the effect on reasoning performance may vary depending on the model, task, and precision level (Frantar, Ashbooks, Hoefler, & Alistarh, 2023) (Xiao, et al.)

This study investigates the relationship between post-training weight quantization and reasoning performance in small instruction-tuned language models. Three models were evaluated: Qwen3-0.6B, Gemma 3 1B IT, and Llama 3.2 1B Instruct. Each model was evaluated using a BF16 reference version and five GGUF quantization formats: Q8_0, Q6_K, Q5_K_M, Q4_K_M, and Q3_K_M.

The central research question is:

Post-Training Weight Quantization in Small Language Models: Reasoning–Efficiency Trade-offs Across GGUF Precision Levels

To investigate this question, the study compares exact-answer accuracy on a fixed 50-question subset of the GSM8K mathematical reasoning dataset, together with latency, peak RAM usage, generation speed, and stored model size. The same questions and evaluation procedure were used across all models and precision levels.

The contribution of this study is an empirical comparison of the accuracy–efficiency trade-off produced by GGUF weight quantization on a CPU-only system. The results provide preliminary evidence about whether reduced precision produces a consistent degradation pattern or whether its effects are model-dependent.

3. Related Work

Weight quantization reduces the numerical precision of model weights to decrease storage and inference costs. Previous work has shown that post-training quantization can compress language models while attempting to preserve their task performance. GPTQ introduced an accurate post-training quantization method for transformer models (Frantar, Ashbooks, Hoefler, & Alistarh, 2023), while AWQ investigated activation-aware weight quantization for efficient large-language-model inference (Lin, Tang, Tang, & Yang). SmoothQuant further addressed activation outliers during quantization (Xiao, et al.). This study differs from these methods because it evaluates the observable reasoning and efficiency effects of llama.cpp GGUF weight quantization across several small instruction-tuned models using the same fixed evaluation set.

4. Research Questions and Hypotheses

This study addresses the following research questions:

RQ1: How does reasoning performance change across different weight-quantization precisions?

RQ2: Does quantization affect mathematical reasoning performance consistently across different small-language models?

RQ3: Does quantization produce a gradual degradation in reasoning accuracy, or is degradation concentrated at particular low-bit precision levels?

RQ4: What trade-off exists between reasoning accuracy and inference efficiency, including latency, generation speed, memory usage, and stored model size?

4.1 Hypothesis

H1: Lower-bit quantization will reduce stored model size and improve CPU inference efficiency compared with BF16 reference models.

H2: Quantization will affect reasoning accuracy, but the magnitude of degradation will vary across model families and precision levels.

H3: The greatest accuracy degradation will occur at the lowest tested precision, Q3_K_M, rather than decreasing uniformly across every quantization level.

5. Methodology

5.1 Experimental Design

This study used a controlled experimental design to examine how post-training weight quantization affects the reasoning performance and inference efficiency of small language models. The study evaluated three officially released small language models: Qwen3-0.6B, Gemma 3 1B IT, and Llama 3.2 1B Instruct. The corresponding model documentation and technical reports were used to identify the model versions and capabilities (Yang, Li, Yang, Zhang, & Hui, 2025) (Team , 2025) (Meta, 2024)

For each model, a reference BF16 GGUF version was created from the original Hugging Face weights. Five quantized variants were then generated using the same llama.cpp quantization pipeline: Q8_0, Q6_K, Q5_K_M, Q4_K_M, and Q3_K_M. Each precision variant was evaluated using the same benchmark questions, textual prompt, decoding configuration, and scoring procedure, while applying each model's appropriate chat template.

5.2 Models

Table 1. Models evaluated in the study

Models	Variants	Publisher
Qwen3-0.6B	Reference and Quantized variants	Qwen Team
Gemma 3 1B IT	Instruction-tuned	Google
Llama 3.2 1B Instruct	Instruction-tuned	Meta

The models were selected because they are relatively small, officially published, and suitable for CPU-based experimentation on a laptop with 8GB of RAM. (Yang, Li, Yang, Zhang, & Hui, 2025) (Team , 2025) (Meta, 2024)

5.3 Dataset

The evaluation used a fixed 50-question pilot subset stored in gsm8k_pilot_50.json. GSM8K was selected because it is a benchmark of grade-school mathematical word problems requiring multi-step reasoning and providing objectively verifiable answers (Cobbe, kosaraju, Bavarian, & Chen, 2021)

The same 50 questions were used for every model and every precision variant.

5.4 Conversion and Quantization

The original Hugging Face model weights were converted into BF16 GGUF format using the llama.cpp *convert_hf_to_gguf.py* tool. The conversion and inference tools were provided by the open-source llama.cpp project (ggml-org).

The BF16 GGUF file served as the reference model for each model family. Quantized versions were generated from that same BF16 GGUF file using llama.cpp's *llama-quantize* utility provided by llama.cpp (ggml-org)

The tested quantization formats were:

- Q8_0
- Q6_K
- Q5_K_M
- Q4_K_M
- Q3_K_M

No independently sourced pre-quantized checkpoints were used.

5.5 Evaluation procedure

All evaluations were performed using llama.cpp on the same CPU-only Windows environment. Each question was evaluated independently using deterministic decoding.

- Temperature: 0
- Maximum new tokens: 256
- Single-turn generation
- Same prompt format
- The appropriate tokenizer and model-specific chat template were used for each model.
- Thinking disabled where supported

The generated answer was extracted and normalized before comparison with the reference numeric answer.

5.6 Evaluation Metrics

The primary metric was exact-answer accuracy:

$$\text{Accuracy} = \frac{\text{Number of correct answers}}{\text{Total number of questions}}$$

Secondary metrics were:

- Average per-question latency
- Peak process RAM usage
- Prompt processing speed
- Generation speed in tokens per second
- Stored model size

The primary analysis focused on whether accuracy degradation was gradual, abrupt, mixed, or absent as numerical precision decreased.

6. Results

6.1 Reasoning Accuracy

Table 2. Accuracy results across models and quantization levels only

Model	BF16	Q8_0	Q_K	Q5_K_M	Q4_K_M	Q3_K_M
Qwen 3 0.6B	54%	46%	42%	48%	54%	24%
Gemma 3 1B IT	42%	32%	36%	36%	44%	40%
Llama 3.2 1B Instruct	34%	36%	38%	34%	32%	20%

Qwen 3 0.6B achieved the highest BF16 accuracy at 54%. Gemma 3 1B IT achieved 42% while Llama 3.2 1B Instruct achieved 34%.

Among the quantized variants, Qwen Q4_K_M achieved 54%, Gemma Q4_K_M achieved 44%, and Llama Q6_K achieved 38%.

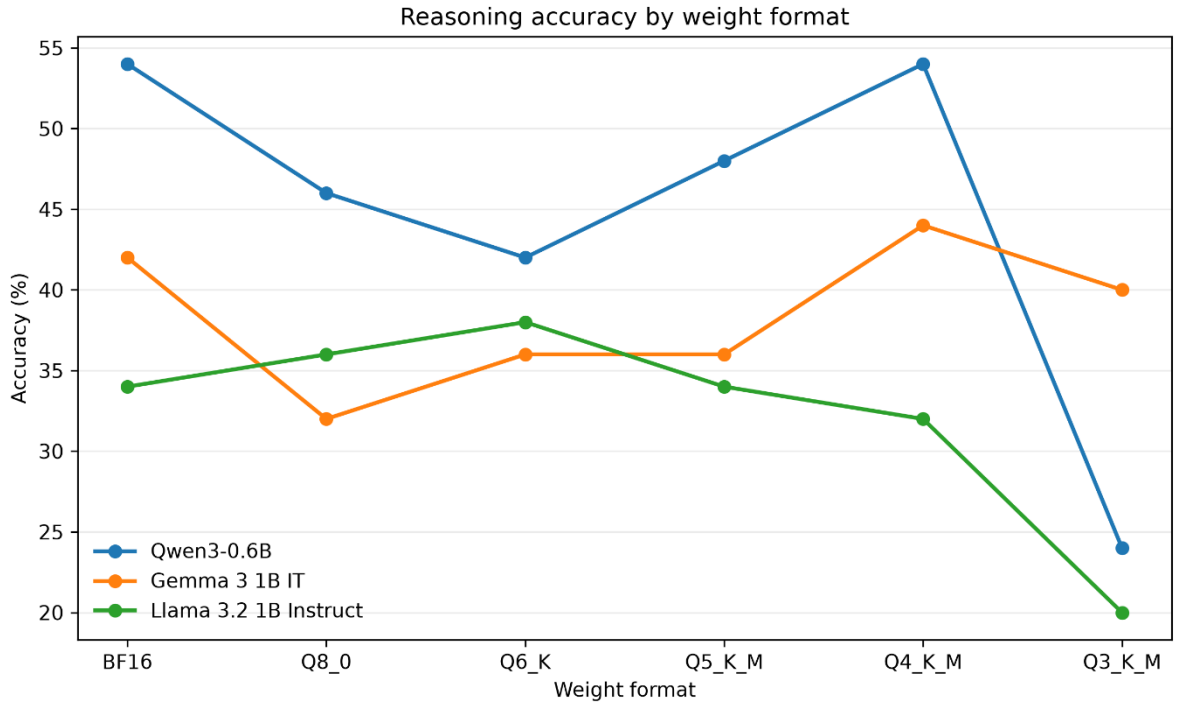


Figure 1. Reasoning accuracy across models and weight-quantization formats

Source: Author's experiments

6.2 Inference Efficiency

Quantization generally reduced latency and increased generation speed.

For Qwen3 0.6B, average latency decreased from 29.72 seconds with BF16 to 21.64 seconds with Q3_K_M. generation speed increased from 11.05 to 25.32 tokens per seconds.

For Gemma 3 1B IT, latency decreased from 36.88 seconds to 16.54 seconds, while generation speed increased from 7.30 to 17.77 tokens per second.

For Llama 3.2 1B Instruct, latency decreased from 59.71 seconds to 26.50 seconds, while generation speed increased from 4.57 to 16.01 tokens per seconds.

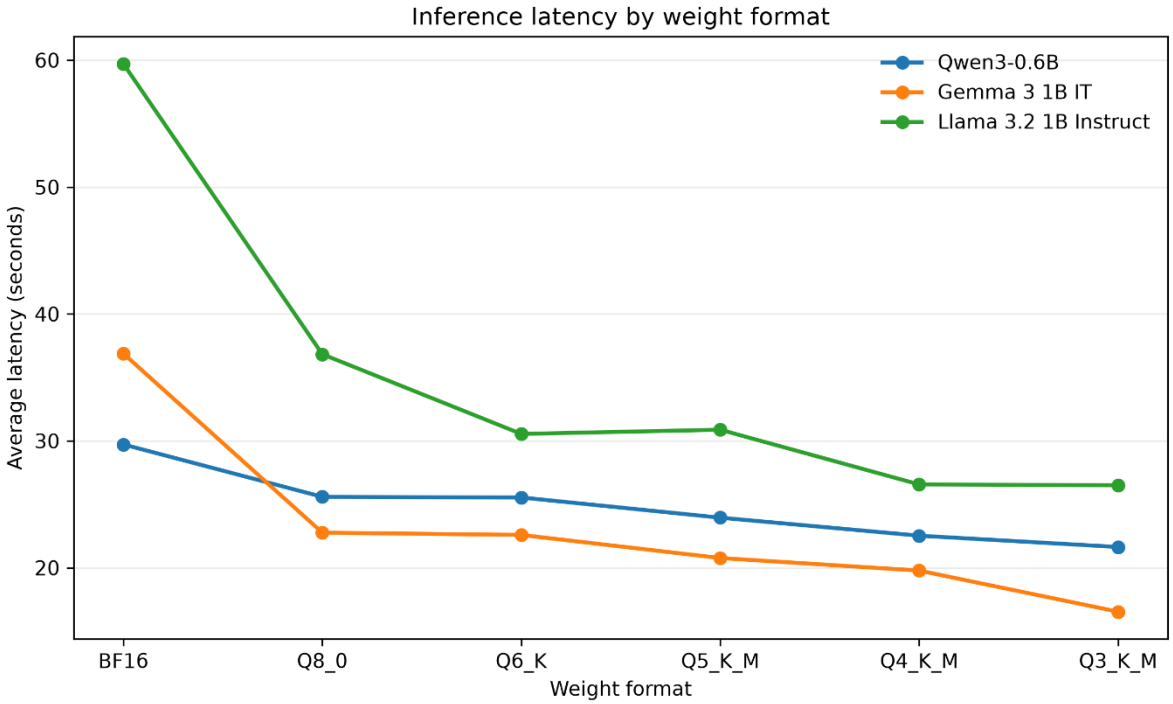


Figure 2. Average inference latency across models and weight-quantization formats

Source: Author's experiments

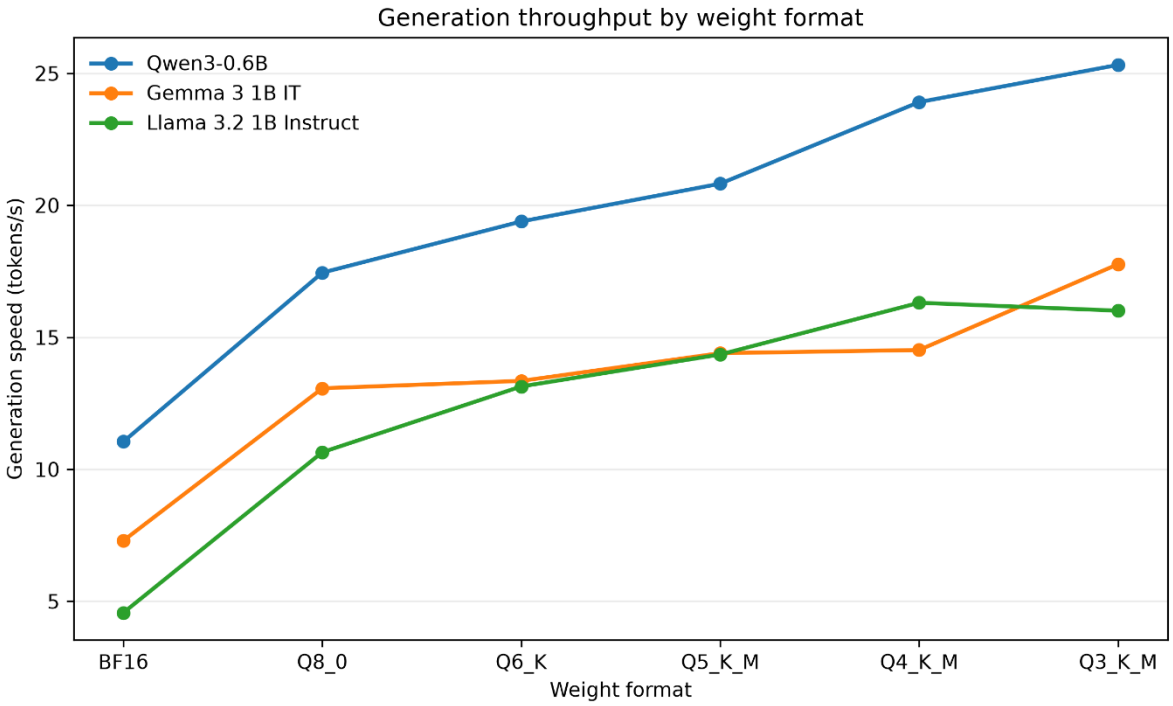


Figure 3. Generation throughput across models and weight-quantization formats

Source: Author's experiments

6.3 memory and Model Size

The stored model size decreased as quantization level decreased. The Q3_K_M variants reduced model size by approximately:

Qwen3 0.6B: 72.6%

Gemma 3 1B IT: 64%

Llama 3.2 1B Instruct: 72.1%

Peak RAM measurements varied across models and did not decreases uniformly at every quantization level.

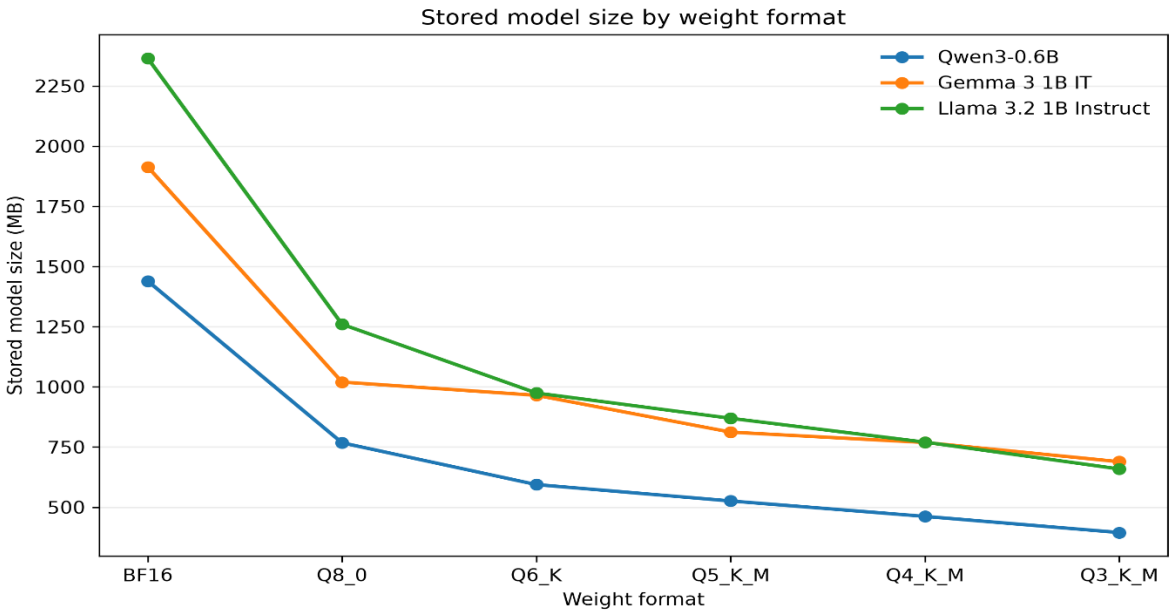


Figure 4. Stored model size across models and weight-quantization formats

Source: Author's experiments

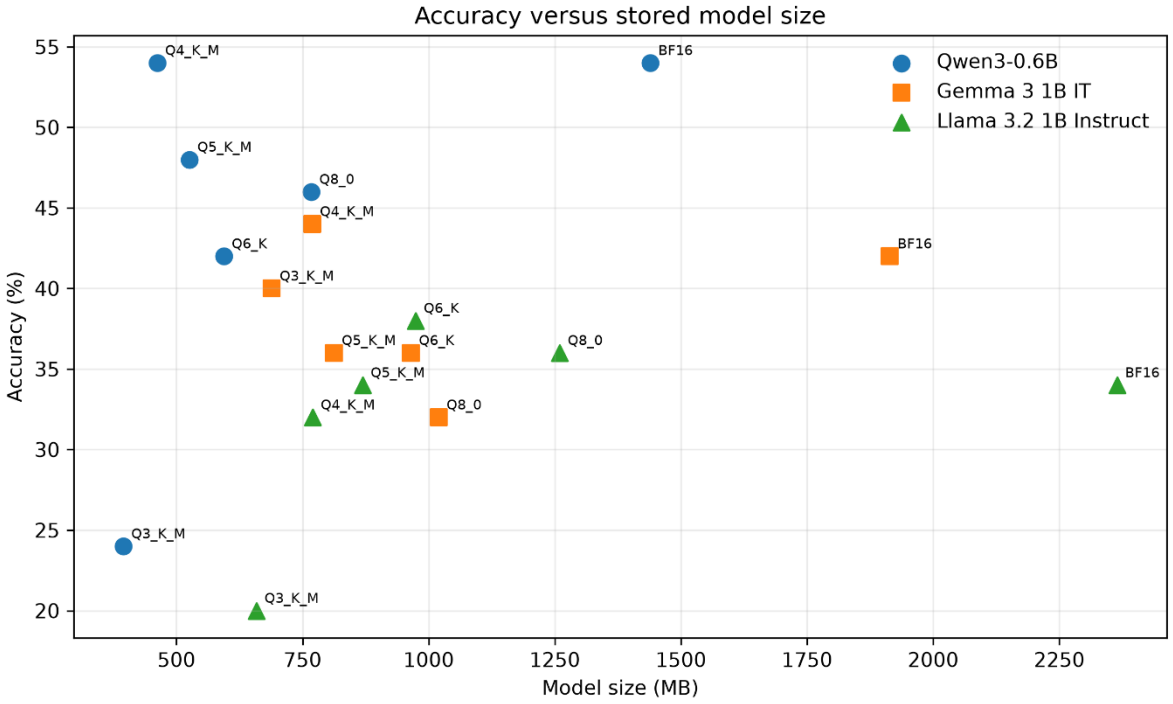


Figure 5. Peak process RAM usage across models and weight-quantization formats

Source: Author's experiments

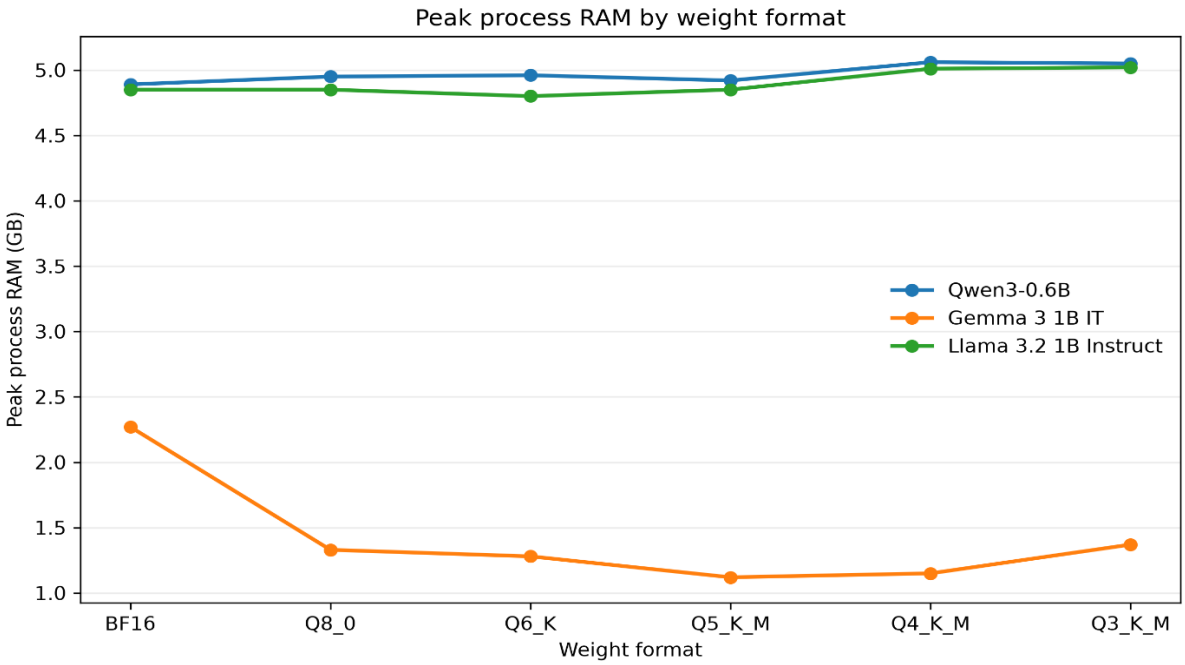


Figure 6. Relationship between reasoning accuracy and stored model size

Source: Author's experiments

6.4 Summary of results

The results show that quantization produced substantial efficiency improvements, but accuracy changes were not uniform across models or precision levels. Some mid-range quantized variants matched or exceeded their BF16 reference accuracy, while the lowest-bit Q3_K_M variants produced larger accuracy changes for Qwen and Llama.

These results described the observed measurements. Their implications and possible explanation are discussed in the next section.

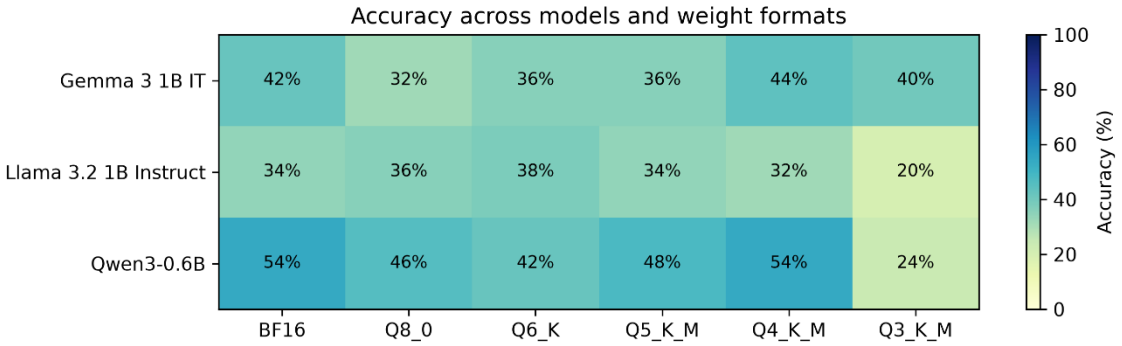


Figure 7. Accuracy comparison across models and weight-quantization formats

Source: Author's experiments

7. Discussion

7.1 Accuracy Degradation Pattern

The results do not show a uniformly gradual decrease in reasoning accuracy as numerical precision decreases. Instead, the effect of quantization varied across model families and precision levels.

For Qwen3-0.6B, Q4_K_M matched the BF16 reference accuracy at 54%, while Q3_K_M decreased to 24%. For Gemma 3 1B IT, Q4_K_M achieved 44%, slightly above the BF16 result of 42%, while Q3_K_M achieved 40%. For Llama 3.2 1B Instruct, Q6_K achieved the highest measured accuracy at 38%, compared with 34% for BF16, while Q3_K_M decreased to 20%.

Therefore, the observed results suggest a mixed and model-dependent degradation pattern rather than a single universal precision threshold.

7.2 Accuracy-Efficiency Trade-off

Quantization produced clear efficiency benefits. The lowest-bit variants generally required less storage and generated tokens faster than the BF16 reference models.

The largest speed improvements were observed for the Llama and Gemma models. Llama Q3_K_M reduced average latency from 59.71 to 26.50 seconds and increased generation speed from 4.57 to 16.01 tokens per second. Gemma Q3_K_M reduced latency from 36.88 to 16.54 seconds and increased generation speed from 7.30 to 17.77 tokens per second.

These results demonstrate that lower-bit quantization can improve practical CPU inference efficiency, although the associated accuracy changes differ by model and quantization level.

7.3 Difference Between Model Families

The three model families produced different accuracy and efficiency profiles. Qwen3-0.6B achieved the highest BF16 accuracy in this evaluation, while Llama3 1B Instruct had the lowest BF16 accuracy but showed substantial latency and throughput improvements after quantization.

Gemma 3 1B IT used less peak process RAM than Qwen and Llama in the recorded experiments. However, absolute accuracy values should not be interpreted as a definitive ranking of the models because the evaluation used only a 50-question pilot subset.

7.4 Answer to the Research Questions

The findings provide the following preliminary answers:

- RQ1: Reasoning performance changed across quantization levels, but the magnitude and direction varied by model
- RQ2: This study focused on mathematical reasoning, so effects across other reasoning categories cannot yet be determined.
- RQ3: The results do not show a single universal degradation pattern. Some models experienced sharp reductions at Q3, while other precision levels produced smaller or mixed changes.

- RQ4: Lower-bit quantization generally improved latency, generation speed, and stored model size, creating a measurable capability-efficiency trade-off.

These findings should be interpreted as preliminary evidence from a controlled 50-question comparison rather than as universal conclusions about all small language models.

8. Limitations

This study has several limitations.

First, the evaluation used only a fixed 50-question pilot subset of GSM8K. the results therefore provide an initial comparison of mathematical reasoning performance but may not generalize to the complete GSM8K dataset or to other reasoning tasks.

Second, each configuration was evaluated once using deterministic decoding. Repeated runs were not performed, so statistical variation and confidence intervals could not be estimated.

Third, all experiments were conducted on one CPU-only Windows laptop with 8GB of RAM. The measured latency, memory usage, and throughput may differ on other hardware or operating systems.

Fourth, the study evaluated only one quantization implementation and five GGUF formats provided through llama.cpp. Other quantization methods, calibration procedures, and runtime may produce different results.

Finally, the three model families differ in architecture, training data, tokenizer, and instruction-tuning procedure. Their absolute accuracy values should therefore not be interpreted as a definitive ranking of overall model quality.

These limitations define the scope of the findings and indicate areas for future work, including larger datasets, multiple reasoning categories, repeated trials, and additional quantization methods.

9. Conclusion

9.1 Conclusion

This study investigated whether post-training weight quantization causes gradual or abrupt degradation in the reasoning capabilities of small language models. Qwen3-0.6B, Gemma 3 1B IT, and Llama 3.2 1B Instruct were evaluated using BF16 reference models and five llama.cpp quantization formats on a fixed 50-question GSM8K subset.

The results show that quantization did not produce one universal degradation pattern. Accuracy changes varied across model families and precision levels. Qwen Q4_K_M matched its BF16 accuracy, Gemma Q4_K_M slightly exceeded its BF16 result, and Llama Q6_K achieved the strongest quantized result for that model. However, Q3_K_M caused larger accuracy reductions for Qwen and Llama.

Quantization consistently provided practical efficiency benefits, including lower model storage requirements, reduced latency, and higher generation speed. These benefits were accompanied by model-dependent changes in reasoning accuracy.

Overall, the findings suggest that quantization can provide useful efficiency improvements, but the choice of precision should consider both computational requirements and the accuracy sensitivity of the specific model.

9.2 Future work

Future research should evaluate larger and more diverse reasoning datasets, including mathematical, logical, commonsense, and multi-step reasoning tasks. Repeated trials could also be used to estimate variability and confidence intervals.

Additional work could compare other quantization methods, calibration-based approaches, larger model families, and different hardware platforms. A larger evaluation would help determine whether the patterns observed in this pilot study generalize beyond the tested models and dataset.

10. References

Cobbe, K., kosaraju, v., Bavarian, M., & Chen, m. (2021). *Training Verifiers to Solve Math Word Problems.* doi:https://arxiv.org/abs/2110.14168

Frantar, E., Ashbooks, S., Hoefler, T., & Alistarh, D. (2023). GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. *arxiv.* doi:https://arxiv.org/abs/2210.17323

ggml-org. (n.d.). *llama.cpp.* Retrieved from https://github.com/ggml-org/llama.cpp

Lin, j., Tang, J., Tang, H., & Yang, S. (n.d.). AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration. *arxiv.* doi:https://arxiv.org/abs/2306.00978

Meta, T. (2024). *llama-models.* Retrieved from https://github.com/meta-llama/llama-models

Team , G. (2025). *Gemma 3 Technical Report.* doi:https://arxiv.org/abs/2503.19786

Xiao, G., Lin , J., Seznec, M., wu, H., Demouth, J., & Han, S. (n.d.). SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models. *arxiv.* doi:https://arxiv.org/abs/2211.10438

Yang, A., Li, A., Yang, b., Zhang, B., & Hui, B. (2025). *Qwen3 Technical Report.* arxiv. doi:https://arxiv.org/abs/2505.09388